2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT) | 979-8-3315-8295-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/PACT65351.2025.00017

Doppeladler: Adaptive Tensor Parallelism for Latency-Critical LLM Deployment on CPU-GPU Integrated End-User Device

Jiazhi Jiang[1,2,#], Xiao Liu[1,3,#], Jiangsu Du[1], Dan Huang[1,*], Yutong Lu[1]
[1]*School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou, China*
[2]*Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai, China*
[3]*Peng Cheng Laboratory, Shenzhen, China*
Email: {jiangjzh6, liux276}@mail2.sysu.edu.cn, {dujiangsu, huangd79, luyutong}@mail.sysu.edu.cn
[#]These authors contributed equally. [*]Corresponding author.

***Abstract*—LLM deployment on end-user devices has attracted significant interest from tech giants and research institutions due to privacy benefits and the elimination of network roundtrips. Reducing latency is crucial for improving user experience. End-user devices, such as desktop and mobile processors, often integrate CPU and GPU on a single die, making tensor parallelism a promising approach to distribute workloads and reduce inference latency. However, the predefined tensor parallelism in traditional practices cannot guarantee optimization due to heterogeneity and resource contention in the CPU-GPU integrated end-user devices. In this paper, we propose Doppeladler, a practical framework designed to facilitate parallel inference of LLM on end-user devices. Doppeladler adaptively optimizes tensor parallelism based on real-time conditions and device status, enhancing resource utilization by refining workload partitioning and scheduling with a heuristic-based approach. Additionally, it lowers the costs of CPU-GPU hybrid execution by managing device resource, trigger workload rebalance during runtime to mitigate the penalty of contention, and minimizing communication overhead through zero-copy techniques and alternate access pattern. Experiments demonstrate that Doppeladler outperforms existing methods by 1.2 to 2.5 times.**

***Index Terms*—LLM inference, Integrated CPU-GPU, Tensor parallelism**

I. Introduction

The past several years have witnessed the tremendous success of LLM (large language model) such as GPT-3[1] and LLaMA[2] in unlocking generative tasks from code generation to question answering. Their current landscapes are increasingly diverse. Due to concerns about preserving data privacy and avoiding network roundtrips, there is a growing demand for their execution directly on end-user devices. Techniques such as model quantization and pruning[3] enable the downsizing of LLM, making it feasible to deploy it on end-user devices and become of interest to many tech giants and research institutions.

Unlike cloud deployments that prioritize throughput, real-world applications on end-user devices typically handle low-frequency, small batch size requests (usually batch size of one), where reducing latency for better interaction is more important than increasing throughput with larger batch sizes. The reduction of inference latency is key factor in enhancing user experience, which directly impacts the responsiveness and interactivity of applications. This creates considerable challenges to the hardware and inference serving infrastructure, given the limited computing capabilities of end-user platforms. Leading hardware vendors, including Apple, Intel and AMD, have introduced various energy-efficient integrated processors with shared main memory accessible by both the CPU and GPU[4], [5], [6]. CPU-GPU integrated devices typically exhibit superior computing capabilities, which provides opportunities to simultaneously utilize CPU and GPU for latency reduction. There are also works that leverage the idea of using both CPU and integrated GPU to enhance workload performance on end-user devices[4], [5], [7].

In terms of distributing deep-learning workload among different devices, there exist three paradigms, namely data parallelism, pipeline parallelism, and tensor parallelism. It is noted that these approaches have distinct impacts on latency and throughput. Data parallelism and pipeline parallelism, both extensively researched in training tasks, primarily focus on enhancing throughput. Conversely, tensor parallelism stands out due to its ability to simultaneously optimize both latency and throughput, making it a highly attractive option. Despite the fact that popular frameworks such as Megatron-LM [8] have proposed and implemented tensor parallelism for transformer models across multiple identical GPUs, they operate based on the assumption of a homogeneous environment and rely heavily on high-bandwidth interconnects, such as NVLink. Consequently, tensor parallelism has been rarely explored or adopted in previous efforts for collaborative processing between CPUs and integrated GPUs (denoted as iGPU).

In previous explorations of GPU and CPU hybrid execution, GPU has always been the first-class citizen. The CPU plays a supportive role, assisting the GPU in various ways. Most of these practices are rooted in the key insight and fact: In discrete CPU-GPU architectures, both CPU and GPU have their own separate memory spaces and are connected via PCIe. CPU memory is larger but has relatively weaker computational capabilities compared to the

979-8-3315-8295-1/25/$31.00 ©2025 IEEE
DOI 10.1109/PACT65351.2025.00017

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

GPU. Tensor parallelism that relies on frequent collective communication can have its performance gains easily offset by the overhead of frequent data transfers. However, in integrated CPU-GPU architectures, the landscape changes substantially. Unlike PCIe-based CPU–GPU systems—where frameworks like Flexgen[9] offload model parameters into CPU memory and overlap computation with communication to mitigate slow PCIe transfers—in an integrated CPU–iGPU device the shared memory space entirely eliminates PCIe overhead. Moreover, because the CPU's compute capability is comparable to that of the iGPU, we can exploit fine-grained tensor parallelism across both processors to reduce inference latency.

Traditional tensor parallelism for LLM inference is suboptimal on integrated CPU-GPU architectures due to its assumptions of homogeneity and rigid parallelism strategies [10], disregarding hardware heterogeneity and diversity leads to workload imbalances and device idleness. Automated tensor parallelism frameworks like Alpa [11] employ optimization algorithms to determine optimal tensor parallelism configurations for training. However, they do not target on integrated CPU-GPU architectures and latency-critical scenarios. The CPU+discrete GPU setups—where the main optimization is boosting throughput by overlapping compute with inherently slow PCIe transfers. What's more, these frameworks only apply when the GPU device operates independently and has access to ample resources. Contentions and constraints such as power limits, memory bandwidth contention, and data exchange overheads exacerbate inefficiencies of directly employing tensor parallelism on integrated CPU-GPU devices, resulting in lower performance compared to single-device execution. Thus, achieving optimal distributed LLM inference on these devices requires a more meticulous approach.

In this paper, we propose Doppeladler, an LLM inference system designed for CPU-GPU integrated end-user devices. Doppeladler employs an adaptive and contention-aware tensor parallelism strategy aimed at minimizing latency during LLM inference. **The key idea of Doppeladler lies in its ability to adaptively and dynamically decompose and schedule LLM operator kernels based on real-time runtime conditions and device status, thereby enhancing device utilization while mitigating co-execution overhead.** Efficiently leveraging tensor parallelism on integrated CPU-GPU architectures for LLM inference poses several challenges: 1) precisely partitioning and scheduling kernels to enhance device utilization, 2) managing resource contention, and 3) minimizing communication overhead between CPU and iGPU.

First, precise partitioning and scheduling of kernels are essential to reduce workload imbalance between the CPU and iGPU. The diversity of end-user processors, along with the heterogeneity between CPU and iGPU, leads to varying kernel execution times in large model inference. As a result, a static and one-size-fits-all approach to tensor parallelism seldom aligns CPU and iGPU execution times. Due to data dependency between operators, synchronization after each operation is required, causing mismatched execution durations that can lead to idle time as the CPU and GPU wait on each other. To achieve more precise partitioning, Doppeladler implements tensor parallelism using a heuristic approach that cleverly leverages the repetitive structure of LLMs, collecting runtime execution information and status from previous layers to refine the partitioning of the operator in subsequent layers.

Second, in integrated CPU-GPU architectures, resource contention arises when executing kernels concurrently on both devices. Although the CPU and iGPU have separate compute resources, they still share resources such as memory bandwidth and power limits. This contention can cause interference, undermining the benefits of tensor parallelism and disrupting workload scheduling. Doppeladler addresses this by adjusting the resource allocation of the device and rebalancing workload distribution to mitigate contention.

Third, tensor parallelism partitions each operator across multiple devices, with each device executing a portion of the computation in parallel. However, due to data dependencies between devices, frequent collective communication is required throughout the execution. This necessitates efficient data sharing and synchronization between the CPU and iGPU. Doppeladler employs zero-copy technique to eliminate data copy on the physical memory space and an alternate memory access pattern to reduce synchronization overhead. Our contributions are summarized as follows:

- We identify the potential device idling and overheads of hybrid execution, such as resource contention on integrated CPU-GPU architecture when applying existing static and predefined tensor parallelism approaches for LLM inference.
- We propose Doppeladler, an LLM inference runtime system designed to optimize tensor parallelism by increasing compute utilization while reducing resource contention on integrated CPU-GPU. The key idea of Doppeladler lies in its ability to adaptively and dynamically decompose operators based on real-time runtime conditions and device status.
- Doppeladler improves device utilization by leveraging the repetitive layer structure of LLMs and using a heuristic method to dynamically optimize workload distribution between CPU and iGPU. It addresses the challenges of heterogeneous device capabilities, varying operator affinities, and real-time hardware status, enabling more effective intra-operator tensor parallelism.
- Doppeladler employs various strategies to lower the costs associated with CPU and iGPU hybrid execution. It mitigates resource contention by managing resource allocation and triggering workload rebalancing while also reducing CPU-iGPU communication overhead through a zero-copy technique and an alternate memory access pattern.

The experimental results show that Doppeladler surpasses baseline methods in most cases and achieves approximately 1.2 to 2.5x latency reduction compared to other methods.

II. Background and Motivation

A. CPU-GPU Integrated End-user Devices

The CPU-GPU integrated end-user devices represent a significant advancement in computing technology, merging the

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

processing capabilities of CPUs and GPUs onto a single chip. In recent years, leading hardware vendors such as AMD, Intel and Apple have developed their own versions of this integrated architecture for end-use and edge devices. It is currently typical in commodity system setups to offer different, heterogeneous, processing devices. Examples include Intel Skylake/Ultra, AMD Radeon, and Apple M-series processors, which integrate CPU and GPU on a single die. As shown in Figure 1, compared to discrete CPU-GPU architectures, often features a shared unified memory between CPU and GPU. Therefore, GPU memory is the host memory on end-user integrated devices. The shared memory eliminates the need for separate memory spaces connected by the PCIe, which was a limitation in discrete CPU-GPU architectures.

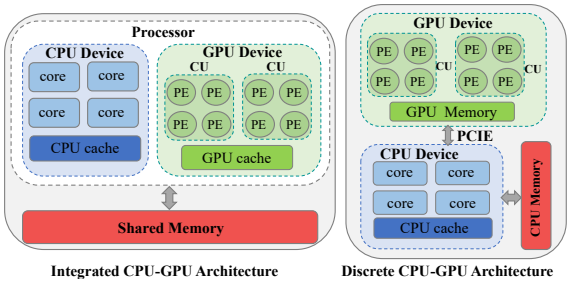

Fig. 1: The illustration of CPU-GPU architectures.

B. Why Tensor Parallelism for CPU-iGPU?

As mentioned in Section I, LLM deployment on end-user devices is characterized by two key factors: **latency sensitivity** and **small-batch processing**. In this context, tensor parallelism offers a clear advantage. By distributing a tensor across multiple devices, it enables simultaneous processing and reducing latency for end-user applications. In contrast, pipeline parallelism improves throughput in high-communication-overhead scenarios but does not reduce latency, as each operator is processed sequentially on a single device. Similarly, data or sequence parallelism is ineffective for latency-critical, small-batch tasks. Thus, tensor parallelism is more suitable for LLM deployment on end-user devices.

C. CPU-GPU Hybrid Execution on Edge Device

We investigate co-execution on CPU–GPU integrated devices for LLM inference. As shown in Figure 2, key operators—Q/K/V matrix generation, self-attention and MLP—are only run on one device, leaving the other idle in the sequential scheme. Inter-operator parallelism (e.g., mapping Q, K and V to separate devices) improves utilization but is limited by data dependencies, since only independent operators can execute concurrently. Intra-operator tensor parallelism offers finer granularity by splitting each operator across devices, yet it typically assumes a homogeneous environment and divides work equally—this can slow the faster device and incurs CPU–GPU memory-bandwidth contention. To minimize idle time and latency, we therefore need an adaptive, heterogeneity-aware scheme that dynamically balances workloads between CPU and GPU while mitigating co-execution overheads.

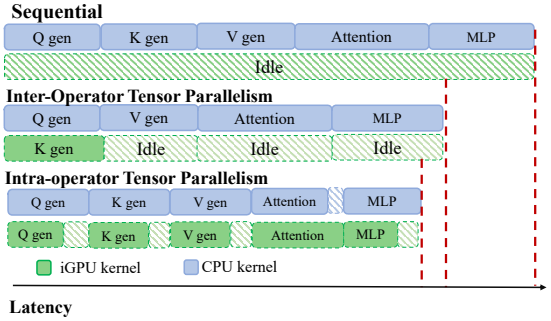

Fig. 2: Approaches for LLM inference on integrated CPU-GPU.

D. Challenges

While tensor parallelism between CPU and iGPU demonstrates promising results in reducing latency for LLM deployment on end-user devices, its implementation can be challenging due to difficulties in achieving appropriate workload partition and scheduling.

1) Widely-varied kernel duration: Figure 3 presents the normalized duration of different kernels in large language models on AMD 5700G processor and the Intel Core Ultra5 processor. Most key modules of LLM inference are all factorized into matrix multiplications in the end [12]. Therefore, general matrix-vector multiplication (GEMV) and general matrix-matrix multiplication (GEMM) are both commonly used operators in decoding and prefill stage of LLM inference separately [13]. As shown in Figure 3, the widely varied kernel durations of LLM inference operators come from two aspects. First, **heterogeneity of hardware capabilities**. The wide range of CPU and iGPU vendors and product types for end-user processors leads to diverse computing capabilities and memory bandwidths. As a result, the duration of LLM operators can vary significantly across different CPUs and integrated GPUs. For instance, the GEMV operator with a dimension size of (2048, 2048, 1) shows nearly identical execution times on both the CPU and iGPU of the Intel Core Ultra 5. However, on the AMD 5700G processor, the iGPU takes approximately 2.5 times longer to execute the same operation compared to the CPU. Second, **the varying operator-device affinity**. Different operators within LLM inference exhibit distinct preferences for either the CPU or iGPU, depending on their computational and memory access patterns. For instance, memory-intensive operators such as GEMV may perform more efficiently on CPUs, while compute-intensive operators such as GEMM may benefit from the parallelism of GPUs.

In a homogeneous device environment, operators can be evenly split without much difficulty. However, the variation of performance on integrated CPU-iGPU complicates the task of model partitioning, making it difficult to devise a uniform, static approach for partitioning and distributing computation tasks across CPU and GPU resources. A one-size-fits-all partitioning scheme is often inadequate for optimizing performance across diverse hardware and operator characteristics to reduce inference latency.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

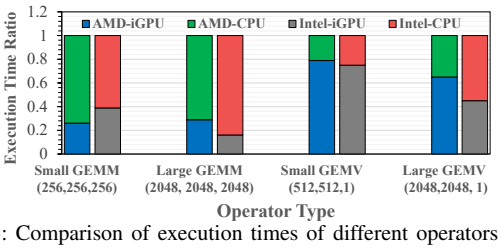

Fig. 3: Comparison of execution times of different operators on the CPU and iGPU of AMD and Intel processors.

2) Resource contention on integrated architecture: While LLM operators can be split to run workloads independently on both the CPU and GPU, their execution can still interfere with each other, especially when competing for computational resources. This competition can alter the balance of CPU and iGPU resources available for LLM co-inference. Moreover, fully utilizing all available resources does not always yield maximum performance due to power constraints and shared memory limitations. (i) **Power constraints.** On power-limited devices like laptops, power allocation between the CPU and iGPU is dynamic. As CPU cores consume more power, the power available for the iGPU decreases, potentially impacting its performance. For example, on a desktop with an Intel Core i7-10750H CPU and a 20W power limit, running large GEMM operations (4096, 4096, 4096) simultaneously on both the CPU and iGPU can result in a 20-30% performance drop on each device. (ii) **Shared memory contention.** When operators on the CPU and iGPU need to access shared memory, it increases bandwidth usage and can cause contention between the devices. This contention can lead to inefficiencies, particularly during memory-bound operations. (iii) **Co-location of other applications.** Unlike server-side environments where LLM inference often run exclusively, end-user devices typically host various other applications alongside LLM serving. This can lead to resource contention, reducing performance gains, and inaccurate estimates of kernel durations, which may result in suboptimal workload partitioning and task allocation.

3) Communication overhead of CPU and iGPU: Although CPU and iGPU physically share memory space, it's partitioned into separate logical spaces. Traditional practices of CPU and iGPU communication models are based on data copy [14]. The CPU and iGPU exchange data by copying it back and forth between their respective memory partitions. When conducting tensor parallelism for LLM inference between CPU and iGPU, the intermediate results of model tensors need to be copied from the logical space of CPU to that of iGPU, or vice versa, for data synchronization and reduction after each operator completes its execution. Although data is not moved over PCIe, which makes CPU-iGPU copying a lot faster, it's necessary to avoid data movement as much as possible. Further, concurrent access by CPU and iGPU also requires the programmer to manage data consistency. Overheads brought by explicit synchronization may affect the overall system performance.

III. The Design of Doppeladler

The key idea of Doppeladler lies in the adaptive and contention-aware tensor parallelism approach which aims to enhance device utilization while reducing hybrid execution overheads for integrated CPU-GPU architecture. It dynamically decomposes the kernels of operators during LLM inference based on the current runtime conditions and device status with a heuristic-based approach, then assigns decomposed kernels to either CPU/iGPU for execution.

Figure 4 demonstrates the overall design of Doppeladler. Upon receiving typically small-batch requests from applications deployed on devices, the requests are forwarded to Doppeladler for processing. The workflow is organized into three primary phases: the profiling and decomposition phase, the execution phase, and the communication phase. The profiling and decomposition phase comprises two key procedures: offline profiling and online decomposition. The offline profiling is executed once before deployment to gather runtime traces, which are subsequently used by the startup predictor in the decomposition procedure. The online decomposition procedure dynamically partitions each operator into two segments (one for CPU and one for iGPU) based on real-time runtime conditions and current device status.

Then the decomposed kernel will be launched and executed in the execution phase. It periodically gathers performance data during the execution of each LLM layer, including kernel execution times on both the CPU and GPU, contention condition of device bandwidth and power consumption. This feedback is then used in the gradient descent of the decomposition phase to refine kernel partitioning for the next layers. The decomposition and execution stage overlap with each other. When the previous operator begins execution, the decomposition of the next operator starts while the gradient descent is already being performed. Finally, the execution results of the decomposed operator kernels from the CPU and iGPU are merged in shared memory using zero-copy during the communication stage.

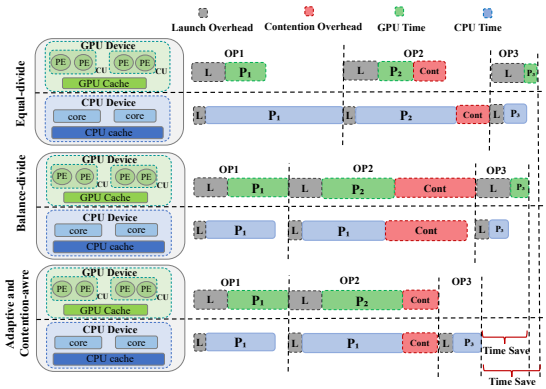

Fig. 5: Conceptual illustration highlighting advantages of adaptive kernel decomposition on integrated CPU-GPU comparing with other tensor parallelism approaches. OP1, OP2 and OP3 generically represent arbitrary operators in LLM.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

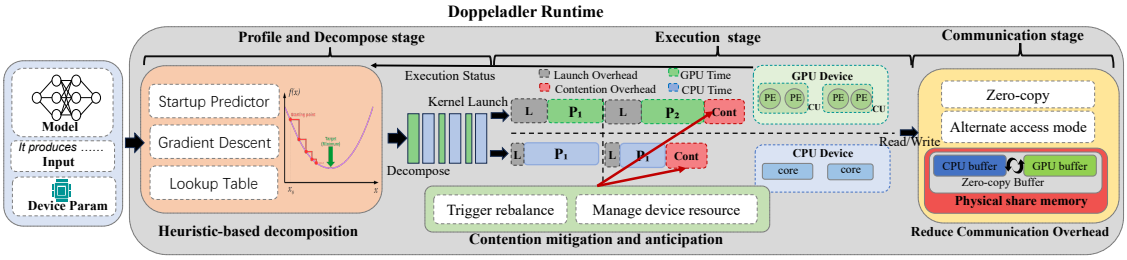

Fig. 4: The design overview of Doppeladler.

A. Adaptive and Contention-aware Parallelism

As shown in Figure 5, several solutions exist for achieving intra-operator tensor parallelism between the CPU and iGPU. The first solution, the equal-division intra-operator model parallelism (used in multi-GPU server settings), evenly distributes operators across devices. However, as discussed in Section II-D, the heterogeneity of the CPU-GPU integrated architecture can lead to load imbalances, causing significant idle time for computing resources, as shown by OP1 and OP2 in Figure 5. The second approach, balance-division, aims to maximize CPU and iGPU utilization by allocating operator kernels based on device capacity. While this ensures better resource saturation, it may not always be optimal. Co-executing memory-intensive operators like GEMV can lead to resource contention, diminishing performance gains, as seen with OP2. Additionally, iGPU's launch overhead means that for short tasks, splitting execution between CPU and iGPU may be less efficient than running solely on the CPU, as illustrated by OP3. The adaptive, contention-aware intra-operator tensor parallelism strategy adjusts execution based on device status to minimize LLM inference latency. For operators like OP1, maximizing device utilization minimizes latency. For OP2, reducing resource usage on the iGPU or CPU can decrease contention overhead, lowering overall latency. Short-duration operators like OP3 are best run solely on the CPU to avoid iGPU launch overhead, where some idle time is acceptable to reduce latency.

B. Heuristic-based Decomposition

As mentioned above, achieving optimal LLM inference performance on integrated CPU-GPU architecture requires an adaptive and contention-aware intra-operator parallelism approach. However, given the variety of operators and devices, customizing for each combination is impractical. To address this, we developed a heuristic workload decomposition algorithm. The heuristic-based decomposition approach leverages the fundamental fact that LLM architectures often consist of dozens or even hundreds of layers stacked with identical structures. This contrasts with CNNs, where each layer's halved features result in varying operator shapes. For instance, in LLaMA2-7B[2], MLP and QKV attention modules, with consistent GEMM dimensions (M, N, K) across layers, account for more than 95% of inference latency. The QK/KV multiplications are identical across all 32 layers, while MLP module, with four identical GEMMs per layer, repeats 128 times across all layers. Therefore, the execution history and conditions of earlier layers can provide valuable insights for the decomposition and scheduling of operators in subsequent layers. As shown in Algorithm 1, the process consists of three phases: the startup phase, the gradient descent phase, and the lookup phase. During the startup phase, due to the lack of historical execution information, we employ a simple execution time estimator based on profiling and collected device data to perform initial workload partitioning and scheduling. In the gradient descent phase, we use runtime execution data to iteratively refine the partitioning and scheduling through gradient descent. This approach corrects the potential partitioning biases introduced by the simple mathematical model used in the startup phase. In the lookup phase, we utilize the optimal scheduling plan derived from the gradient descent phase and monitor performance fluctuations. This enhances adaptability to complex dynamic conditions, such as variations in the available power budget.

1) Startup phase: In the startup phase, our goal is to obtain an optimized partition based on the limited execution information. The execution history of all operators is stored in the history table, as shown in Table I. Each operator is encoded into an operator ID based on its operation, data type, and tensor shape, and operators with the same ID are treated as identical. Among the records in the table, the optimal record indicates the configuration and the corresponding execution time that yielded the fastest result. Other records include details of the most recent execution, execution solely on the CPU, and execution solely on the iGPU. In each record, the percentage of the operator allocated to the CPU and iGPU is denoted as $\mathtt{P_{CPU}}, \mathtt{P_{iGPU}}, 0 \leq \mathtt{P_{CPU}} \leq 1, \mathtt{P_{iGPU}} = 1 - \mathtt{P_{CPU}}$, and the execution time of each device after division is $\mathtt{T_{CPU}}, \mathtt{T_{iGPU}}$.

As observed in Section II-C, balancing the load between the CPU and iGPU during operator partitioning can reduce execution stalls, thereby improving overall performance. Furthermore, different operators have varying affinities for different hardware. To achieve these goals, in the startup phase, we use the execution times of the CPU and iGPU individually to identify the operator's hardware preferences while balancing the workload.

First, we collect the execution time of the operator on the CPU alone, denoted as $\mathtt{T_{cpu}}$, and the execution time on the iGPU alone, denoted as $\mathtt{T_{igpu}}$. Then, we use the profiled data

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Heuristic-based decomposition

```
Input: OP the description of the operator; H the operator's execution history
Output: P the percentage of the operator allocated to the CPU
Initialization: H.OptRound the optimization round in execution history table; PerfFlucT performance fluctuation threshold;
if H is not initialized then
    StartUp();
else if H.OptRound ≤ OptRoundLimit then
    GradientDescent();
else
    LookUp();
return P

Function StartUp():
    if H.iGPUOnly is not initialized then
        P ← 1;
    else if H.CPUOnly is not initialized then
        P ← 0;
    else
        T_cpu, T_igpu ← H.CPUOnly, H.iGPUOnly;
        P ← T_igpu ÷ (T_cpu + T_igpu);
    H.OptRound ← H.OptRound + 1;
End Function

Function GradientDescent():
    T_optimal, P_optimal ← H.OptimalRecord;
    T_latest, P_latest ← H.LatestRecord;
    Grad ← (T_optimal − T_latest) ÷ (P_optimal − P_latest);
    Step ← DecreaseStep(Step, H.OptRound);
    if Grad ≤ GradThreshold then
        P ← P_optimal;
    else
        P ← P_optimal − Grad · Step;
    H.OptRound ← H.OptRound + 1;
End Function

Function LookUp():
    T_optimal, P_optimal ← H.OptimalRecord;
    T_latest, P_latest ← H.LatestRecord;
    if P_optimal = P_latest then
        if PerfFluc(T_optimal, T_latest) ≥ PerfFlucT then
            RestartFromStartUp();
            return
    P ← P_optimal;
End Function
```

along with Equation 1 to get the partition plan $\mathtt{P}_{\mathtt{CPU}}$.

$$\mathtt{P}_{\mathtt{CPU}} = \frac{\mathtt{T}_{\mathtt{igpu}}}{\mathtt{T}_{\mathtt{cpu}} + \mathtt{T}_{\mathtt{igpu}}} \tag{1}$$

The Equation 1 balances the load between the CPU and iGPU based on their execution time, where the tasks allocated to each device are inversely proportional to the device performance obtained through sampling. For operators with hardware affinity, typically those with low computational demand or complex logic, there is a significant difference in $\mathtt{T}_{\mathtt{cpu}}$ and $\mathtt{T}_{\mathtt{igpu}}$, often spanning several orders of magnitude, due to overheads like kernel launch on the iGPU. Equation 1 can identify affinity through values of $\mathtt{P}_{\mathtt{CPU}}$ that are close to either 0 or 1.

2) Gradient descent phase: The startup phase relies on limited sample data and simple formula that contain inevitable biases. As a result, the workload partitioning during the startup phase differs from the optimal approach, leaving room for further refinement. Therefore, Doppeladler adopts a heuristic-based approach, which uses historical execution data of the same operator under different partitioning configurations to approximate the optimal partitioning configurations quickly.

Current large language models, such as LLaMA [2], are based on the Transformer architecture and repeat the same operators across different layers in the model. This repetition provides ample opportunities for optimization using a heuristic-based approach, allowing us to derive better partitioning and scheduling strategies for each operator before the completion of a single inference process. Doppelader calculates the gradient of the operator's execution time with respect to the CPU partition ratio, using both the operator's historical optimal execution records and the current execution results. Based on this gradient, Doppelader determines the next partition P_{CPU}. During the gradient descent phase, Doppelader uses the latest partitioning configurations provided by Algorithm 1. To improve the stability and effectiveness of the gradient descent algorithm [15], we employ a decreasing step size and limit the number of optimization iterations. Once the number of optimization iterations reaches an experimentally determined threshold, the algorithm stabilizes and adopts the optimal partitioning according to the optimal record from execution history table.

3) Lookup phase: After obtaining the optimal model parallel configurations of various operators in the gradient descent phase, Doppelader saved it in the execution history table as shown in Table I. The profiling and recording of execution history are accomplished by embedding time-measurement code directly into the operator execution logic within source code of Doppeladler. Our experiments indicate that the gradient descent phase generally converges to the optimal decomposition configuration by approximately the 10th layer of the model. LLMs deployed on devices typically contain dozens of identical layers. For the remaining layers after the gradient descent phase, Doppeladler retrieves the optimal decomposition configuration from the execution history table with one request. With the operator ID, each operator can look up the optimal configurations in the table for workloads of different requests. The lookup table is reused across calls and updated per operator to log execution data. The execution history table uses the LRU algorithm to periodically discard historical execution data. Storing all combinations of tensor shapes, operator types, and configurations would be impractical, and as the device's state changes, outdated decomposition strategies may no longer be optimal.

C. Contention Mitigation and Anticipation

Doppeladler simultaneously utilizes the CPU and iGPU to improve device efficiency and reduce inference latency. However, resource contention may introduce overhead on each device, prolonging kernel execution and preventing the system from achieving its performance targets despite parallel execution. Contention is closely related to the specific runtime status of the devices. For instance, in end-user devices with limited cooling capacity, such as laptops, running high-load

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Execution history table for storing parallism configuration and performance of the LLM operators.

operator ID	optimization round	optimal record	latest record	cpu only record	igpu only record
OP1	1	$(P_{CPU}, P_{iGPU}, T_{CPU}, T_{iGPU})$	$(P_{CPU}, P_{iGPU}$ ...)	$(1, 0, T_{CPU}, 0)$	(...)
OP2	3	$(P_{CPU}, P_{iGPU}, T_{CPU}, T_{iGPU})$	$(P_{CPU}, P_{iGPU}$ ...)	$(1, 0, T_{CPU}, 0)$	(...)
OP3	15	$(P_{CPU}, P_{iGPU}, T_{CPU}, T_{iGPU})$	$(P_{CPU}, P_{iGPU}$ ...)	$(1, 0, T_{CPU}, 0)$	(...)
.....	.....	.....	.....	.....	.....

tasks for extended periods can lead to device overheating, causing a reduction in the power of processors and, consequently, affecting the computational performance of the CPU and iGPU. Additionally, other applications may compete for computational resources on device, resulting in changes to the proportion of CPU and iGPU resources available for inference.

Doppeladler does not monitor individual resources with specific tools for contention but instead detects it by comparing the execution times of operators to a threshold empirically determined through profiling contention patterns and analyze execution logs on integrated CPU-GPU platforms. Exceeding this threshold signals potential contention, based on the observation that most contention scenarios lead to latency anomalies. Doppeladler maintains a historical record of operator execution under normal conditions. When contention is detected, a noticeable performance degradation occurs when applying refined decomposition strategies from the execution history table, as compared to prior runs.

Doppeladler handles the contention problem from two aspects: (i) manages resource allocation of kernels and (ii) anticipates performance degradation in profile and decompose stage. Doppeladler first mitigates the issue of resource contention by reducing redundant resource allocation for the launched kernels. In the OpenCL programming model, resource requirements for kernels are predefined at the time of programming, making it difficult to adjust iGPU resource usage dynamically. To alleviate resource contention, Doppeladler initially reduces the number of CPU threads and the core utilization for kernels executed on the CPU. While this approach effectively lowers contention overhead, it may also increase execution latency for CPU tasks due to the reduced resource availability. As a result, the decomposition configurations stored in the execution history table become outdated. The diminished CPU resources can lead to an imbalanced workload distribution, causing under-utilization of the iGPU. To address this, Doppeladler ensures precise operator kernel decomposition by re-initiating the heuristic-based decomposition process, allowing the operator's decomposition configuration to be adjusted dynamically.

D. Reduction of Communication Overheads

Having the result data located on the main shared memory does not guarantee direct access by both the CPU and iGPU. Usually, there's a hardware configured split for GPU and GPU memory. When conducting tensor parallelism for LLM inference between CPU and iGPU, the intermediate results of model tensors need to be copied from the logical space of CPU to that of iGPU as shown in Figure 6, leading to significant communication overhead. The zero-copy mechanism is one feature of integrated CPU-GPU architecture. It can help eliminate redundant data transfer through a memory-mapping mechanism. The key hardware feature that enables zero-copy is the fact that the CPU and GPU have shared physical memory. The OpenCL API *clCreateBuffer()* is used to allocate a buffer in OpenCL unified memory. With flag *CL_MEM_ALLOC_HOST_PTR*, it asks OpenCL to allocate the buffer on the host memory instead of VRAM on devices, since the integrated GPU shares host memory. While OpenCL provides zero-copy APIs, implementing efficient zero-copy for tensor parallelism requires significant efforts to cooperate with our parallel methods.

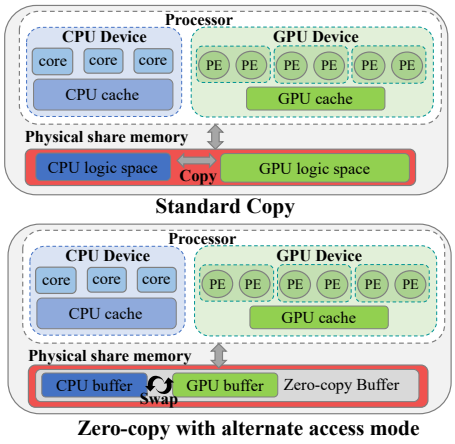

Fig. 6: Reduction of communication overheads.

Doppeladler enhances standard zero-copy mechanisms through an alternate access pattern to eliminate explicit synchronization. With the zero-copy mechanism, data copying is eliminated, enabling the CPU and iGPU to access the same pinned memory concurrently. This simultaneous access necessitates the inference runtime to ensure data consistency, which requires effective synchronization and careful memory management. To mitigate the consistency overhead associated with zero-copy, Doppeladler implements two measures. First, as illustrated in Figure 6, the allocated zero-copy buffer is further subdivided into two distinct regions. Unlike the standard copy practice, these regions share the same unified memory space but are managed by the runtime to ensure that the CPU and iGPU access them in an alternating pattern. The CPU first reads from and writes to its designated region, while the iGPU performs read and write operations on its own region. Subsequently, the roles are reversed, with the CPU and iGPU accessing the opposite regions. The main idea is to reduce simultaneous access to the same memory space by both the CPU and iGPU devices. This approach allows Doppeladler to avoid explicit synchronization during the merging of results.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

Second, in Doppeladler's GEMM operator decomposition strategy, column-parallelism is preferred over row-parallelism. As shown in figure 7, this column-parallel approach, combined with the zero-copy buffer's separate CPU and GPU regions, enables the final result gathering without requiring the CPU and iGPU to access the same memory space. This design effectively eliminates data consistency overhead.

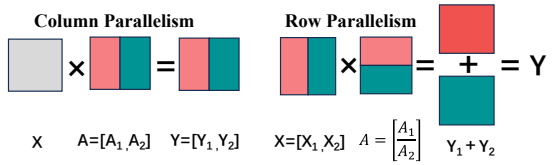

Fig. 7: Matrix multiplication kernel decomposition.

IV. Implementation and Evaluation

We implement a prototype system for Doppeladler with about 5k lines of code in C++. Since we take on-device LLM as the serving target, we reuse OpenCL kernels primarily from a highly optimized LLM inference system, llama.cpp[16], to build models. In this section, we evaluate the serving ability of Doppeladler prototype under a variety of models, platforms, and tasks.

A. Experimental Setup

TABLE II: Model Specifications.

Model Name	Parameters	Layers	Heads	Hidden Size	Precision
Llama2	7B	32	32	4096	Q4/Q8/FP16
Baichuan2	7B	32	32	4096	Q4/Q8/FP16

Node testbed. We deploy Doppeladler on two desktop devices. One is equipped with AMD 5700G CPU and 32GB DDR4 memory. Another is equipped with Intel Core Ultra5 with 48GB DDR5 memory. In CPU-GPU integrated architectures on end-user platforms, CPU and iGPU are typically from the same vendor. AMD 5700G integrates AMD Vega8 GPU, Intel Core Ultra5 integrates Intel Arc GPU. Each CPU is typically paired with a corresponding integrated GPU in integrated architectures. iGPU/CPU share the host memory.

Model setup. Since the Transformer is the most common backbone for large models, we choose two large Transformer models with 7B parameters. Each model is available in three precision variants—Q4, Q8, and FP16—for testing purposes.

Metrics. As CPU-GPU integrated architecture is more common in local deployments, such as desktops, which prioritize low latency, the primary metric for our evaluation is the average latency to process a single input/output token (tokens/ms). This metric is calculated by dividing the total number of processed tokens by the aggregate end-to-end response time.

Baselines. We compare Doppeladler with both the CPU and GPU versions of llama.cpp, the state-of-the-art inference system for end-user devices, as well as two intra-operator tensor parallelism approaches widely used in other DL inference frameworks.

- **CPU-only/iGPU-only sequential inference.** Current state-of-the-art inference systems for end-user devices, such as llama.cpp[16], adopt serial execution, where all operators are run exclusively on either the CPU or the iGPU, without leveraging tensor parallelism to utilize multiple devices simultaneously. We used both the CPU and GPU versions of llama.cpp for our evaluation.
- **Equal-divided parallelism.** As noted in section III-A, the equal-divide intra-operator model parallel scheme evenly splits operator kernels across the CPU and GPU. It assumes a homogeneous device environment. This approach is widely used for multi-GPU parallelism, as in Megatron-LM[17]. We implement this scheme with llama.cpp for comparison.
- **Balance-divided parallelism.** As noted in section III-A, balance-divided intra-operator tensor parallelism is a heterogeneity-aware workload partitioning approach. It statically pre-decomposes tasks based on the devices' computation capabilities and has been adopted in works such as EdgeNN [4]. We implement this scheme with llama.cpp for comparison.

B. General Performance Evaluation

In this section, we compare Doppeladler against baseline methods on randomly generated traces with sequence lengths ranging from 64 to 1024. Throughput is usually not the main concern for on-device LLM inference, as the focus is on delivering personalized services. Consequently, we typically use a small batch size of 1. We conducted 10 rounds of experiments and then took the average of the results. Detailed results are presented in Figure 8.

In all cases shown in Figure 8, Doppeladler presents significant advantages in inference latency. Specifically, Doppeladler achieves a 1.2 to 1.9x latency reduction compared to CPU execution and a 1.3 to 2.5x reduction compared to iGPU execution. It also outperforms other model parallel approaches, with 1.4 to 2.2x lower latency than equal-division and 1.3 to 1.6x lower latency than balanced-division. As we can see in Figure 8, when the sequence length is small, and the model uses Q4 or Q8 quantization, placing operators on the CPU has a clear advantage over the iGPU. The reason is that many LLM inference operators are ultimately factorized into matrix multiplications, whose dimensions are proportional to the sequence length and batch size[12]. For workloads with small sequence lengths and a batch size of 1, the computational intensity is low, and the launch and data conversion overhead (Q4/Q8 is not supported on the iGPU adopted in experiments) on the iGPU become dominant. As the sequence length increases, the advantages of executing on the iGPU become more pronounced. Therefore, directly using static tensor parallelism schemes, such as equal-division or balanced-division, can be less effective than simply running inference on the CPU due to their suboptimal workload partitioning. This causes the iGPU execution to lag behind and become a bottleneck. These methods only achieve significant advantages over serial execution when the sequence length is

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

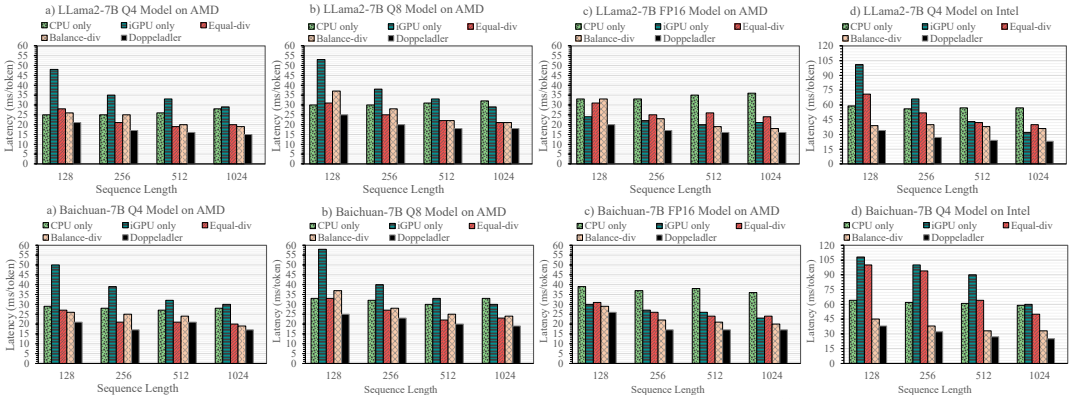

Fig. 8: Latency results as sequence length increases with randomly generated traces with different models, platforms and precision.

large. In contrast, Doppeladler dynamically adjusts workload partitioning and scheduling based on device and operator characteristics. Therefore, regardless of sequence length, model, and hardware platform, Doppeladler consistently outperforms serial execution and static methods like equal-division and balanced-division. Additionally, Doppeladler's efforts to reduce GPU-CPU communication and resource contention costs enable efficient use of both the CPU and iGPU compared to traditional tensor parallelism approaches.

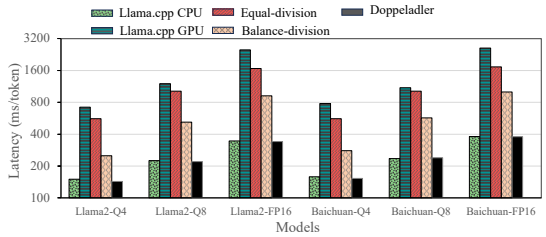

Fig. 9: The latency results of generative task. The sequence length of the starting point is 64.

C. Evaluation on Generative Tasks

Generative tasks are important workloads in large model inference, involving two distinct execution phases: the prefill phase and the decoding phase [18]. The prefill phase performs similarly to general tasks in section IV-B that takes a sentence as input and outputs the encoded representation as the starting point for generating a response. The decoding phase is unique because it generates one token at a time, gradually forming a complete sentence. This section focuses on the decoding stage, employing the KV cache technique [19]. We perform 32 iteration of the decoding stage constantly with a sequence length of 64 as the starting point and a batch size of 1 on the AMD platform. As shown in Figure 9, for the decoding stage of generative tasks, Doppeladler achieves a modest performance improvement of under 5% compared to CPU serial execution, yet significantly outperforms other methods. The decoding stage of the generative task is less suitable for execution on the iGPU, and its latency is significantly higher—up to five times that of the CPU.

The primary reason for this is the introduction of KV Cache in the decoding stage, which substantially reduces the computational intensity of the operators. As a result, the launch overhead and data type conversion costs on the iGPU exceed the operators' actual execution time. As described in Section III-A, forcibly offloading a fraction of these operators to the iGPU in such cases would increase latency and degrade overall performance. Therefore, Doppeladler adaptively executes most decoding stage operators on the CPU. In contrast, static methods like equal-division and balanced-division still split the operators, leading to slower execution on the iGPU. Specifically, equal-division is over four times slower than Doppeladler, and balanced-division is more than twice as slow. This does not indicate that the efforts of Doppeladler is less effective in the decoding stage compared to the CPU-only solution. Instead, it further shows that Doppeladler can adaptively choose the optimal model partitioning based on actual device and operator characteristics. On platforms with lower GPU launch overhead, partitioning decoding stage to is still promising to reduce significant latency.

Therefore, end-to-end latency reduction of Doppeladler depend on the downstream task. For tasks such as text summarization (long input, short decoding), acceleration follows the pattern in Figure 8. For tasks such as text generation (short input, long decoding), the improvement is less significant compared to CPU version of llama.cpp, while still significantly outperforms other solutions.

D. Evaluation on Heuristic Decomposition

Figure 10 illustrates the heuristic workload decomposition process of Doppeladler with a GEMM operator during Llama-7B inference on the AMD platform. The sequence length is 128, and the batch size is 1. During the inference process, Doppeladler profiles the execution time of the GEMM operator on both the CPU and GPU using layers 1 and 2. In layer 3, the startup predictor described in section III-B makes an initial partition based on the profiling of the first two layers, allocating 60% of the workload to the CPU. From layer 4 to layer 10, the gradient descent algorithm iteratively refines the workload partition with the goal of minimizing latency. After

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

layer 10, the algorithm converges, determining that allocating 70% of the workload to the CPU results in the minimum latency for the operator. This partition configuration is then saved in the execution history table, and subsequent layers use this configuration for the GEMM operator. The effectiveness of heuristic decomposition stems from repetitive nature of LLM operators mentioned in Section III-B.

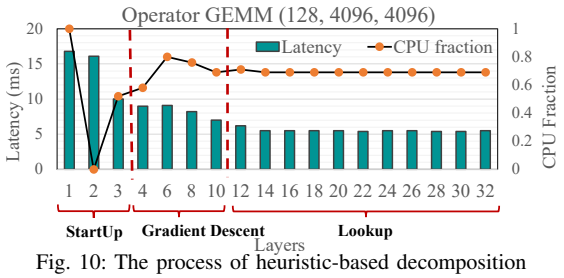

Fig. 10: The process of heuristic-based decomposition

E. Benefits of Contention Mitigation

Figure 11 illustrates how Doppeladler mitigates contention and triggers rebalancing with a GEMM operator during Llama-7B inference on the AMD platform. As we can see, the execution history table stores decomposition configurations from previous executions, allowing layers 1 to 7 to use historical configurations. However, in layer 8, Doppeladler detects a sudden increase in the operator's execution latency, exceeding a predefined threshold and indicating contention. In response, in layer 8, Doppeladler first reduces the number of CPU cores in use and then re-triggers the heuristic-based decomposition to adjust the workload distribution. Compared to maintaining configuration unchanged during contention, measures taken by Doppeladler have resulted in a 25% reduction in latency.

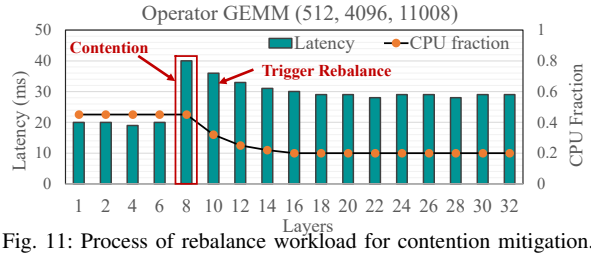

Fig. 11: Process of rebalance workload for contention mitigation.

The frequency of retriggering decomposition depends on runtime device status. In our experiment, if the system dynamically adjusts its CPU/iGPU frequency to balance power consumption and thermal stress, it triggers rebalancing once in our experiment. If persistent resource contention occurs during subsequent inferences, Doppeladler remains in a stable process (empirically validated in Figure 10 and 11) that achieves near-optimal parallel scheduling.

F. Benefits of Zero-Copy Mechanism

This section presents the benefits of reducing communication overheads, as discussed in Section III-D. We conducted experiments with LLM inference using both the standard copy mechanism and the zero-copy mechanism with an alternating access pattern. The evaluation was performed on the AMD platform using Llama-7B with Q4 precision, a batch size of 1, and varying sequence lengths. As shown in Figure 12, the zero-copy mechanism reduces communication overhead by approximately 10% compared to the standard copy mechanism. The main performance gain comes from reducing the data copy overhead when merging CPU and iGPU computation results. The separate CPU and iGPU regions in the zero-copy buffer, combined with the alternate access pattern, also reduce the overhead of maintaining data consistency.

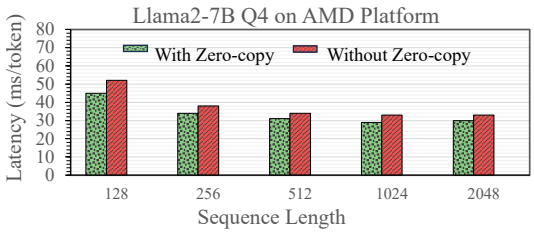

Fig. 12: The effect of reducing communication overheads.

G. Overhead Analysis and Evaluation

Overheads of offline profiling: Offline profiling involves a single on-device inference while recording its performance data, so the duration is the model's inference time on target hardware. This process only needs to be performed once at deployment.

Overheads of decomposition: The complexity of Algorithm 1 itself has a time complexity of O(1), involving only straightforward mathematical calculations and hash table lookups. During inference, Algorithm 1 is executed N times, where N is the number of operators in the LLM. Experimental results show that the overhead of one round of LLM inference is limited to a maximum of 1.8 ms, accounting for less than 1% of the LLM execution time. With further decomposition and computation overlap mentioned in Section III, its impact on inference latency becomes negligible, as confirmed by the observed performance improvements. The historical table typically contains only hundreds of records, occupying about 20MB of memory, with negligible querying overhead.

Overheads of contention mitigation: Doppeladler doesn't monitor individual resources for contention. Instead, it detects contention by comparing operator execution times against a threshold, empirically set at 1.2 times the expected duration. Beyond the threshold, Doppelader assumes contention occurs. This is based on the fact that most contention leads to latency anomalies, as discussed in Section III-C. Once contention occurs, it's detected as soon as a recorded operator completes execution. Therefore, contention detection incurs minimal overhead, involving only a straightforward time comparison. Rebalancing overhead corresponds to the cost of recomputing the decomposition, as detailed above.

Accuracy evaluation: According to our evaluations across different precision settings, running an LLM on a GPU-only or CPU-only setup, compared to Doppeladler for the same downstream task, yields slightly different scores on iGPU

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

and CPU due to precision differences inherent to vendor's hardware. According to experiments, the computation result exhibits discrepancies between 1e-4 and 1e-3, with LLM text generation remains identical with execution on single device, confirming hardware discrepancy are negligible.

H. Portability of Optmization Techniques

Doppeladler's adaptiveness is evaluated across four request lengths, two models, three precision levels, two hardware platforms, and two inference stages, as shown in Figures 8 and 9. Since most end-user devices feature integrated CPU–iGPU architectures, we conduct experiments on representative systems. Although the methods are portable to other integrated processors, fine-tuning is recommended for optimal performance. The heuristic decomposition can be adjusted during gradient descent to adapt to new processing unit configurations by updating the operator ratio in the lookup table. Contention mitigation based on latency fluctuation and zero-copy techniques enabled by OpenCL are generally applicable across platforms with unified memory. While Doppeladler achieves over 5% improvement for generative tasks compared to CPU-only execution in certain setups, its main advantage lies in adaptively partitioning workloads to exploit CPU–iGPU co-running efficiency, particularly on platforms with balanced resources and low kernel launch overhead.

Most large language models are built on Transformer architecture where adaptive tensor parallelism is widely applicable. (1)Our techniques generalize to non-KVCache discriminative models (e.g., intent classification on edge devices), achieving similar latency reductions as shown in Fig.9; (2)Doppeladler's optimization is also orthogonal to advanced methods like speculative decoding and mixture-of-experts, which still rely on the Transformer backbone and can further benefit from our adaptive tensor parallelism.

V. Related Work

Runtime system for LLM inference. State-of-the-art systems like llama.cpp coarsely distribute layers between CPU and GPU memories, effectively using both during inference. While this practice can reduce GPU resource requirements, such as GPU memory in discrete CPU-GPU architectures, it doesn't alleviate inference latency due to data dependencies between layers. Similarly, PowerInfer [13] leverages the inherent locality in LLM inference by assigning minor "hot" neurons to the GPU, while the majority of "cold" neurons are managed by the CPU. Hepti[10] enables parallel inference of transformer-based DNN models across multiple edge devices, emphasizing dynamically adapting to the network status connecting these devices. Flexgen[9], TwinPilots[20] and [21]are all throughput-oriented LLM inference systems for PCIe-connected CPU–GPU architectures, but they pursue very different goals. They focus on techniques to overlap computation and data communication to hide the slow PCIe transfer latency in CPU–GPU collaborative inference and improve throughput without sacrificing latency. Doppeladler targets the latency-critical scenario for integrated CPU-GPU architecture. It proposed adaptive tensor parallelism to reduce inference latency and handle unique challenges of integrated CPU-iGPU such as widely-varied kernel duration and on-chip resource contention, which is mostly orthogonal to works designed for PCIe-based architecture.

ML-IR frameworks: ML-IR frameworks and ML compliler such as TVM[22], XGen and LiteRT provide operator fusion and auto-tuning for CPUs or GPUs, but none orchestrate dynamic co-execution across an CPU-GPU integrated architecture. They don't support hybrid tensor parallism between CPU-iGPU for reducing inference latency.

Integrated CPU-GPU scheduling and execution. There has been a large amount of prior work focusing on improving the device utilization on integrated CPU-GPU architecture. EdgeNN[4] and CoDL[23] perform parallel CNN inference on heterogeneous mobile processors. FineStream[7] performs window-based fine-grained workload scheduling between CPU and GPU to take advantage of both architectures for stream processing. Dopia[5] is a fully automated framework that improves the performance of data-parallel workloads by adjusting the degree Of parallelism on integrated architectures. However, These solutions were designed specifically for LLM inference they fall short in understanding transformer architecture and LLM inference workload.

Tensor parallelism for LLM. Due to memory limitations, tensor parallelism is employed to train large models that exceed the capacity of a single GPU. This strategy can be broadly divided into intra-operator parallelism [8], [24], [25], [26], [27], [12] and inter-operator parallelism [28], [29], [30]. Techniques such as MPress [31] and Mobius [32] exploit NVLink interconnects to offload tensors to GPUs with spare memory while hiding transfer costs during training. Alpa [11] automates inter- and intra-operator parallelism for distributed training, and AlpaServe [33] extends this to automated pipeline parallelism for inference throughput on GPU clusters. In contrast, Doppeladler takes an orthogonal approach by leveraging the repetitive structure of LLMs to dynamically implement adaptive tensor parallelism, reducing latency on enduser devices.

VI. Conclusion

This paper presents Doppeladler, an LLM inference runtime system for end-user devices with adaptive, contention-aware tensor parallelism. It integrates a heuristic-based decomposition, resource contention anticipation, and communication overhead reduction techniques for dynamic decomposition and scheduling of LLM kernels on CPU-GPU architectures. Evaluations demonstrate that Doppeladler outperforms other solutions, reducing latency by 1.2 to 2.5 times across different platforms while enhancing device utilization and minimizing co-execution overheads.

VII. Acknowledgments

This research was supported by supported by Guangdong S&T Program under Grant No. 2024B0101040005, and supported in part by the Start-up Fund of Beijing Normal University (NO.28700-312200502503).

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] Antonio Polino, Razvan Pascanu, and Dan Alistarh. Model compression via distillation and quantization. *arXiv: Neural and Evolutionary Computing,arXiv: Neural and Evolutionary Computing*, Feb 2018.

[4] Chenyang Zhang, Feng Zhang, Kuangyu Chen, Mingjun Chen, Bingsheng He, and Xiaoyong Du. Edgenn: Efficient neural network inference for cpu-gpu integrated edge devices. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pages 1193–1207. IEEE, 2023.

[5] Younghyun Cho, Jiyeon Park, Florian Negele, Changyeon Jo, Thomas R Gross, and Bernhard Egger. Dopia: online parallelism management for integrated cpu/gpu architectures. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 32–45, 2022.

[6] Ismet Dagli and Mehmet E Belviranli. Shared memory-contention-aware concurrent dnn execution for diversely heterogeneous system-on-chips. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 243–256, 2024.

[7] Feng Zhang, Lin Yang, Shuhao Zhang, Bingsheng He, Wei Lu, and Xiaoyong Du. Finestream: Fine-grained window-based stream processing on cpu-gpu integrated architectures. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 633–647, 2020.

[8] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, et al. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.

[9] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.

[10] Juhyeon Lee, Insung Bahk, Hoseung Kim, Sinjin Jeong, Suyeon Lee, and Donghyun Min. An autonomous parallelization of transformer model inference on heterogeneous edge devices. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pages 50–61, 2024.

[11] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.

[12] Jiazhi Jiang, Jiangsu Du, Dan Huang, Zhiguang Chen, Yutong Lu, and Xiangke Liao. Full-stack optimizing transformer inference on arm many-core cpu. *IEEE Transactions on Parallel and Distributed Systems*, 34(7):2221–2235, 2023.

[13] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456*, 2023.

[14] Mirco De Marchi, Francesco Lumpp, Enrico Martini, Michele Boldo, Stefano Aldegheri, and Nicola Bombieri. Efficient ros-compliant cpu-igpu communication on embedded platforms. *Journal of Low Power Electronics and Applications*, 11(2):24, 2021.

[15] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

[16] ggerganov. llama.cpp, 2023.

[17] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[19] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624, 2023.

[20] Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference. In *Proceedings of the 17th ACM International Systems and Storage Conference*, pages 91–103, 2024.

[21] Xuanlei Zhao, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices. *Proceedings of Machine Learning and Systems*, 6:162–172, 2024.

[22] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.

[23] Fucheng Jia, Deyu Zhang, Ting Cao, Shiqi Jiang, Yunxin Liu, Ju Ren, and Yaoxue Zhang. Codl: efficient cpu-gpu co-execution for deep learning inference on mobile devices. In *MobiSys*, volume 22, pages 209–221, 2022.

[24] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.

[25] Shenggan Cheng, Ziming Liu, Jiangsu Du, and Yang You. Atp: Adaptive tensor parallelism for foundation models. *arXiv preprint arXiv:2301.08658*, 2023.

[26] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, et al. Mesh-tensorflow: Deep learning for supercomputers. *Advances in neural information processing systems*, 31, 2018.

[27] Jiazhi Jiang, Jiangsu Du, Dan Huang, Dongsheng Li, Jiang Zheng, and Yutong Lu. Characterizing and optimizing transformer inference on arm many-core processor. In *Proceedings of the 51st International Conference on Parallel Processing*, pages 1–11, 2022.

[28] Yanping Huang, Youlong Cheng, Ankur Bapna, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 103–112, 2019.

[29] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. Terapipe: Token-level pipeline parallelism for training large-scale language models. In *International Conference on Machine Learning*, pages 6543–6552. PMLR, 2021.

[30] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 1–15, 2019.

[31] Quan Zhou, Haiquan Wang, and et al. Mpress: Democratizing billion-scale model training on multi-gpu servers via memory-saving inter-operator parallelism. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 556–569. IEEE, 2023.

[32] Yangyang Feng, Minhui Xie, Zijie Tian, Shuo Wang, Youyou Lu, and Jiwu Shu. Mobius: Fine tuning large-scale models on commodity gpu servers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 489–501, 2023.

[33] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, 2023.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

APPENDIX

A. Abstract

We provide the **ComboLLM/Doppeladler** artifact to evaluate adaptive, contention-aware intra-operator tensor parallelism for latency-critical LLM inference on CPU–GPU integrated end-user devices. The artifact contains (1) a fork of `llama.cpp` with an OpenCL backend and Doppeladler runtime extensions (heuristic decomposition, contention mitigation, and zero-copy alternate access), and (2) scripts to automate experiments and collect results. It allows reproducing the evaluation in Section 4 of the paper across models, precisions, sequence lengths, and hardware platforms.

B. Artifact check-list (meta-information)

- **Algorithm:** Adaptive, contention-aware intra-operator tensor parallelism with heuristic-based kernel decomposition and zero-copy communication.
- **Model:** LLaMA2-7B and Baichuan2-7B (Q4, Q8, FP16). Dummy weights for functional tests; trained weights not redistributed.
- **Run-time environment:** Linux (tested on Ubuntu 22.04 LTS). Build via CMake or GNU Make; OpenCL runtime and CLBlast required for iGPU acceleration.
- **Hardware:** Integrated CPU–GPU devices with unified memory (tested: AMD 5700G (Vega8), Intel Core Ultra5 (Arc iGPU)); RAM $\geq$ 32 GB recommended (48 GB used on Intel tests).
- **Metrics:** Latency per token (ms/token) for prefill and decoding; auxiliary microbenchmarks for GEMM/GEMV.
- **Output:** Output and CSV logs under `experiments/eval/`.
- **Experiments:** Reproduce Figs. 8–12 in Section 4.
- **Disk space:** $\sim$10–20 GB for binaries, GGUF models, and logs.
- **Prep time:** $\sim$1–2 hours (toolchain, OpenCL/CLBlast install, build).
- **Experiment time:** minutes to $\sim$2 hours depending on model/seq length.
- **Publicly available:** Yes; code and Zenodo snapshot (see A.3.1).
- **Code license:** Derivative of `llama.cpp`/ggml licenses (see repository headers).

C. Description

1) How to access: Source code and scripts are packaged with the submission. A public snapshot for AE is published on Zenodo: 10.5281/zenodo.16791568. The development repository is available at https://github.com/JIANGJZ/Doppeladler. Pre-built binaries are optional; we provide build instructions in A.4.

2) Hardware dependencies:

- CPU–iGPU integrated platform with unified memory (AMD or Intel).
- Adequate system RAM (32–48 GB recommended).

3) Software dependencies:

- CMake $\geq$ 3.18 or GNU Make; C++17 compiler (GCC $\geq$ 9 / Clang $\geq$ 10).
- OpenCL headers & ICD loader, vendor OpenCL runtime.
- **CLBlast** development package.
- Python 3.8+ with `numpy`, `sentencepiece`, `gguf` (optional for helpers).

4) Models: We use GGUF formats of LLaMA2-7B and Baichuan2-7B (Q4/Q8/FP16). We do not redistribute trained weights. For functional checks, dummy weights can be used; for full reproduction, provide your own GGUF models compatible with `llama.cpp`.

D. Installation

The following steps are validated on Ubuntu 22.04.

1) Install prerequisites:
```
sudo apt-get update
sudo apt-get install -y build-essential \
cmake git pkg-config ocl-icd-opencl-dev \
clinfo libclblast-dev
```
2) Build (CMake, Release, with CLBlast):
```
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release \
      -DLLAMA_CLBLAST=ON ..
cmake --build . -j
```
3) (Alternative) GNU Make:
```
make LLAMA_CLBLAST=1 -j
```
4) (Optional) Python helpers:
```
pip install -r requirements.txt
```

E. Experiment workflow

Device selection. Choose OpenCL device by

```
export GGML_OPENCL_DEVICE='Your device name'
```

Quick sanity.

```
# quick test
GGML_OPENCL_DEVICE='Your device name' \
./build/bin/combo_simple \
    -m /path/to/llama-2-7b.Q4_K_M.gguf \
    'what is GPT?' 0

# benchmark
GGML_OPENCL_DEVICE='Your device name' \
./build/bin/combo_bench \
    -m /path/to/llama-2-7b.Q4_K_M.gguf \
    1 128,256,512 0 1 0 10000
```

End-to-end evaluation (collect CSV).

```
# Change **DEVICE_NAME, COMBO_BENCH_PATH, \
  MODEL_BASE, MODELS_ARGS**
# in **experiments/eval/eval.py** \
according to your environment
# before executing the script.
```


Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.

```
python experiments/eval/eval.py \
    --out-dir /path/to/result
```

Microbenchmarks.

```
python .../AMD_5700G/mbenchmark.py
python .../CoreUltra5/mbenchmark.py
```

F. Evaluation and expected results

On the devices listed in A.3.2, Doppeladler reduces latency by 1.2–2.5 $\times$ compared to CPU-only/iGPU-only and outperforms equal/balanced division across sequence lengths and precisions, consistent with Section 4 (Figs. 8–12). Absolute numbers vary with vendor drivers and power limits; the relative trends should match the paper.

G. Experiment customization

All runs accept flags for model path, precision, sequence length, threads, and device selection. See `experiments/benchmark_llama7B/benchmark.sh` and `experiments/eval/eval.py`. Modify model sets or output directories to extend configurations.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 16,2026 at 08:24:15 UTC from IEEE Xplore. Restrictions apply.